

Bayt Al Hekma (بيت الحكمة) Library Management System

Complete Program Description

A full account of what the program is, what it models, how every part behaves, and why it is built the way it is.

1. What This Program Is

The Bayt Al Hekma Library Management System is a console application that replaces a community librarian's wooden card index.

It does two jobs. The first is **circulation**: keeping track of a catalogue of borrowable items, who currently holds each one, what state each item is in, and how long each may be kept. The second is **finances**: working out what a late return costs, applying the correct waiver for the member's category, adding an administrative charge that is deliberately not waivable, and recording the debt against the member so it can be paid off later.

The program is built around a single idea: **the library holds items that are similar enough to be handled together, but different enough that each must be allowed to answer for itself**. Every item can state how long it may be kept and what it costs to return late, but no two item types answer either question the same way. The program is arranged so that the code which loops over the catalogue never needs to know which specific kind of item it is holding.

Everything else in this document follows from that one idea.

2. The Building Blocks

This section describes every class: what it is responsible for, what it holds, what it can do, and what it deliberately refuses to do.

3.1 ItemStatus — an enumeration

Four values: AVAILABLE, ON_LOAN, RESERVED, LOST.

An item is always in exactly one of these states and there is no fifth possibility. Using a fixed set of values rather than free text means an invalid state is impossible to write down — a misspelling becomes a compile error rather than a bug discovered at the desk.

3.2 MembershipType — an enumeration that carries data

Three values, each holding its own waiver rate. Unlike ItemStatus, this is not merely a set of labels: each value is an object that knows its own rate and can be asked for it.

The consequence is that the rest of the program contains no table mapping category names to percentages. The category *is* the rate. Adding a fourth category is a one-line change in one file.

3.3 Renewable — an interface

Two capabilities:

- **Renew the loan** — attempt to extend it, and report whether it worked
- **Report the renewal limit** — how many times this item may be renewed in total

This is a contract about what an object can *do*, not about what an object *is*. Books and magazines sign this contract; DVDs do not.

The distinction matters. All three types are library items — they share an identity. Only two of them can be renewed — they share a capability. Modelling the first with inheritance and the second with an interface is what allows a DVD to be a perfectly ordinary catalogue item in every other respect while remaining impossible to renew.

Had renewability been modelled as a *kind* of item instead, the hierarchy would have had to split before it began, and every shared behaviour would have been duplicated on both sides of the split.

3.4 Member

Holds: a name, a membership ID that never changes, a category that never changes, a balance owed, and a count of items currently held.

Can: report all of these; have its name corrected; be charged a fine; accept a payment; report whether it is currently allowed to borrow; record a borrowing; record a return.

Refuses: to let anyone set the balance or the item count directly. There are no such operations. Both numbers move only through controlled paths:

- **Charging a fine** rejects any amount that is not positive, then adds it.
- **Paying a fine** rejects any amount that is not positive, rejects any amount larger than the balance owed — a member cannot overpay — and only then deducts it. It reports back whether it succeeded.
- **Recording a borrowing** increases the count by one.
- **Recording a return** decreases the count by one, and refuses to go below zero.

The borrowing eligibility check is worth noting separately. A member may borrow only if they hold fewer than three items *and* owe no more than 100 EGP. The member

answers this question itself rather than having the library inspect its fields and decide. The rule about who may borrow lives with the thing the rule is about.

Two constructors exist: a new member starting from nothing, and a member carried over from the old card index who already owes money. The first defers to the second rather than repeating the setup, so there is exactly one place where a member is actually assembled.

3.5 `LibraryItem` — an abstract class

Holds, shared across every item type: a permanent catalogue ID, a permanent title, a current state, a borrower name, and a renewal count. It also holds three facts that belong to the library rather than to any individual item: the library's name, the administrative charge, and a running count of how many items have ever been catalogued.

Can:

- Report all of its properties
- Be marked reserved or lost, and brought back
- **Lend itself to a member** — but only if it is currently available. If it is on loan, reserved, or lost, the attempt fails and says so.
- **Take itself back** — reset the state, clear the borrower, reset the renewal count
- **Record a renewal** against the current loan
- **Display itself** as one line: ID, category, title, state, borrower, loan period, and the fine for being one day overdue

Cannot be created directly. `LibraryItem` is abstract. Asking for “just a library item” is a question with no answer — there is no rule for how long a generic item may be kept or what it costs to return late — so the program refuses to let anyone ask.

Demands of every item type three answers the parent cannot supply:

- What does it cost to return this item a given number of days late?
- How many days may this type be kept?
- What is this category called?

The parent knows every item *has* an answer to all three. It has no idea what any of them are.

Two design commitments worth stating explicitly:

The display method is written once. All three item types display identically, and that shared method calls all three of the demanded methods — none of which the parent implements. This works because at the moment the display runs, the object is never a bare `LibraryItem`; it is always some specific kind, and that kind's answers are the ones that execute.

Taking an item back cannot be overridden. An item type is free to invent its own fine rule and its own loan period. It is not free to redefine what returning means. If a type could override the return behaviour and forget to reset the state, that item would remain permanently on loan — and nothing would report an error. It would simply vanish from the library's available stock, quietly, forever. Preventing that is not a security measure; it is the original designer stating that this particular behaviour is load-bearing.

3.6 The three item types

Each extends `LibraryItem`, passes the ID and title up to the parent, adds the extra fields it needs, and answers the three demanded questions.

Book. Adds an author and a page count. May be kept 14 days. Costs 5.00 EGP per overdue day, without limit. Renewable up to twice.

Magazine. Adds an issue number. May be kept 7 days. Costs 3.00 EGP per overdue day, **but the total stops at 30.00 EGP** however long it is kept. Renewable once.

DVD. Adds a runtime. May be kept 3 days. Costs 15.00 EGP per overdue day, without limit. Not renewable.

The magazine's cap is the single most instructive line in the program. Where the book and the DVD differ from each other only by a number, the magazine differs by *shape* — its rule contains a maximum and therefore cannot be expressed as “the daily rate times the days” at all. A design that centralised the fine calculation and parameterised the rate would have to grow a special case here. The design used instead does not notice, because it never assumed the three rules were related in the first place.

3.7 Library

The library holds two collections — the catalogue and the member register — each with a count of how many entries are actually in use.

Registering things. An item is refused if the catalogue is full or its ID is already taken. A member is refused if the register is full or the membership ID is already taken.

Finding things. Two searches: an item by catalogue ID, and a member by membership ID. Both return nothing rather than failing when there is no match, and every caller is expected to handle that.

Listing things. Three listings: the whole catalogue; items filtered by state; and all members with what each one owes.

Calculating things. Four figures:

- **Items on loan** — how many are currently out
- **Loan rate** — items on loan as a percentage of the catalogue

- **Total outstanding** — every member's balance, added up
- **Projected fines** — a budgeting figure the head librarian asks for monthly: if every item currently on loan came back a given number of days late, what would the library be owed before waivers?

That last figure is where the polymorphism becomes visible as a number rather than a listing. It asks every item on loan the same question, and a book, a magazine, and a DVD each answer differently — with the magazine's answer capped. One loop, three rules, no type checking.

Operations. Three composite actions:

Lending. Find the item and the member; refuse if either is missing, if the item is not available, or if the member is not eligible. On success, record the borrower on the item and increase the member's held count.

Processing a return. Find the item; refuse if it is not on loan or the overdue count is negative. Find the member **from the item's recorded borrower name — and this must happen before the item is taken back**, because taking it back clears that name. Then calculate the fine, charge it, decrease the member's held count, print the breakdown, and free the item.

Attempting a renewal. Find the item and report if it does not exist. Then ask whether the item has the renewal capability at all. If it does, attempt the renewal — which can still fail if the item is not on loan or has already reached its limit. If it does not, explain that this item type cannot be renewed.

This capability check is the single place in the entire program where an object's type is examined at runtime. It is permitted because of what it asks. It does not ask *"is this a book or a magazine?"* — it asks *"can this object be renewed?"* The first question would mean the design had failed; the second is the entire reason interfaces exist.

Reporting. One report covering both halves of the system: catalogue size, items ever catalogued, items on loan, the loan rate, total outstanding fines across all members, and the projected fine figure.

3.8 Main

Builds the library, populates it with items and members, then loops: print the menu, read a choice, dispatch it, repeat until the user exits.

One detail is worth pointing at. Three different kinds of item are created and all of them are handed to the same collection. From that moment on, the library holds a set of items and has no record of which specific kind each one was — and never needs one.

7. Every Operation In Detail

The program presents ten operations plus an exit.

1 — View catalogue. No input. Loops over every item and prints one line each: ID, category, title, state, borrower, loan period, and one-day fine. Three item types produce three different loan periods and three different fines from one loop.

2 — Register member. Asks for a name, membership ID, and category. Refuses a duplicate ID or a full register. Creates the member with a zero balance.

3 — Borrow item. Asks for an item ID and a membership ID. Refuses if either is missing, if the item is not available, or if the member holds three items or owes more than 100 EGP. On success, records the borrower and confirms the loan period.

4 — Return item. Asks for an item ID and how many days overdue, where zero means on time. Refuses if the item is not on loan or the count is negative. Calculates the fine in the fixed order, charges it, prints an itemised breakdown, reports the member's new balance, and frees the item.

5 — Renew loan. Asks for an item ID. Checks first whether the type can be renewed at all, explaining if not. If it can, attempts the renewal, which may still fail if the item is not on loan or has hit its limit. Reports how many renewals remain.

6 — Search item by ID. Prints that one item's details, or reports it was not found. Never crashes.

7 — View items by status. Asks which state, then lists only items in it, or reports there are none.

8 — Pay outstanding fines. Asks for a membership ID and an amount. Refuses a non-positive amount and refuses an overpayment. Reports the new balance.

9 — View all members. Lists every member with name, ID, category, items held, and balance owed.

10 — Library report. Catalogue size, items ever catalogued, items on loan, loan rate, total outstanding across all members, and the projected fine figure for a five-day overdue scenario.

0 — Exit. Ends the loop.

8. Every Rule The Program Enforces

Catalogue rules

1. Catalogue IDs are unique; a duplicate is refused at registration
2. A catalogue ID and title never change once the item is created

3. Lending is only possible from the available state
4. An item marked reserved or lost cannot be lent
5. Returning is only possible from the on-loan state
6. Returning always resets the state, borrower, and renewal count — and no item type may redefine this

Member rules

7. Membership IDs are unique; a duplicate is refused at registration
8. A membership ID and category never change
9. A member may hold at most three items at once
10. A member owing more than 100.00 EGP cannot borrow
11. A fine charged must be positive
12. A payment must be positive and must not exceed the balance owed
13. The balance can never become negative
14. The count of items held can never become negative
15. Neither the balance nor the count can be set directly by any means

Fine rules

16. Days overdue must not be negative
17. Zero days overdue means nothing is charged, including no administrative charge
18. The magazine fine never exceeds 30.00 EGP however long the item is kept
19. The waiver applies to the base fine only, never to the administrative charge
20. The member must be identified from the item before the item is freed

Renewal rules

21. Only item types carrying the renewal capability can be renewed
22. An item can only be renewed while it is on loan
23. An item cannot be renewed beyond its type's limit
24. Attempting to renew a type that cannot be renewed produces an explanation, not a crash and not silence

General rules

25. Neither collection may exceed its capacity
26. Searching for something that does not exist reports it and never crashes
27. Every operation returns to the menu rather than terminating the program